

CI/CD Pipeline Documentation

1. Project Overview

This project focuses on building an end-to-end CI/CD pipeline for a simple Flask web application using GitHub, Docker, GitHub Actions, Terraform, Amazon ECR, and AWS ECS (Fargate). The goal is to automate the build and deployment process from code push to production deployment.

Objective

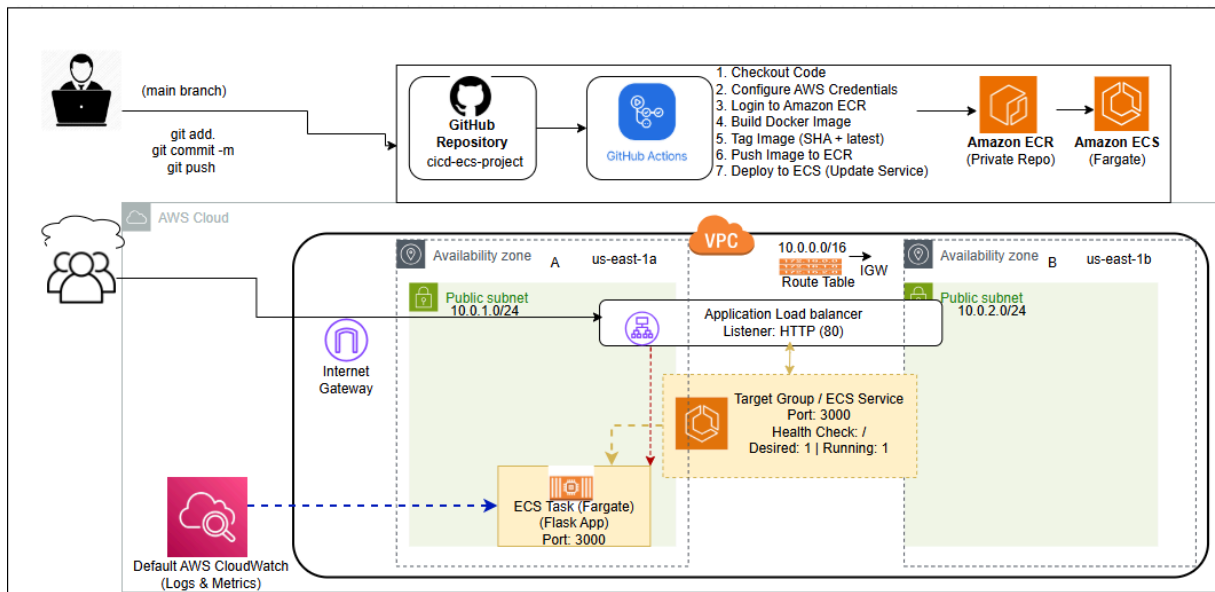
Build and deploy a simple Flask web application with an automated CI/CD pipeline that:

- Builds the application
- Containerizes the application using Docker
- Provisions infrastructure using Terraform
- Pushes Docker images to Amazon ECR
- Deploys containers to AWS ECS using GitHub Actions

Tech Stack

- Python Flask
- Docker
- Git & GitHub
- GitHub Actions
- Terraform
- AWS ECS (Fargate)
- Amazon ECR
- Application Load Balancer

Architecture



2. Git Initialization and Version Control

Commands Used

```
git init
git branch -M main
```

Git configuration was set using:

```
git config user.name "Thompson"
git config user.email "onwubikothompson@gmail.com"
```

Initial commit:

```
git add .
git commit -m "Initial project structure"
```

Explanation

Git was initialized to enable version control from the beginning of the project.

Using version control ensures:

- Change tracking
- Collaboration
- Rollback capability
- Integration with GitHub Actions

The main branch was used because the CI/CD pipeline will later trigger automatically on pushes to this branch.

3. Application Development

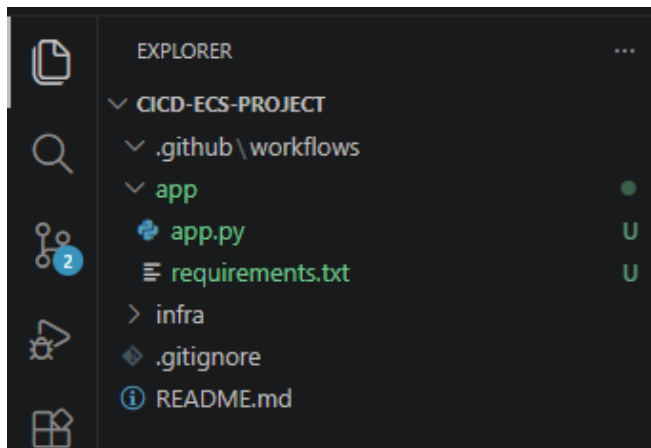
Objective

Develop a simple Flask web application that returns:

Hello from CI/CD Pipeline!

Application Files

The application files were created inside the app directory:



app.py

```
from flask import Flask

app = Flask(__name__)
```

```
@app.route("/")
def home():
    return "Hello from CI/CD Pipeline!"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=3000)
```

requirements.txt

```
Flask==3.0.3
```

Explanation

Flask was selected because it is lightweight and easy to set up.

The application exposes a single route:

- "/" → Returns the required response

The application runs on port 3000 as required by the challenge.

4. Environment Setup and Troubleshooting

Challenge Encountered

Python was initially not installed on the local machine.

Attempting to verify Python returned errors:

```
python --version
python3 --version
```

Both commands failed.

Resolution

Python 3.12 was installed using Chocolatey.

Command used:

```
choco install python312 -y
```

Python installation was verified using:

```
python --version
```

Output:

```
Python 3.12.10
```

Lessons Learned

Using Chocolatey simplifies software installation and aligns with DevOps automation practices by making environment provisioning reproducible.

5. Dependency Installation and Local Application Testing

Install Dependencies

Command used:

```
pip install -r app/requirements.txt
```

This installed Flask and all required dependencies.

Run Application

Command used:

```
python app/app.py
```

Application output:

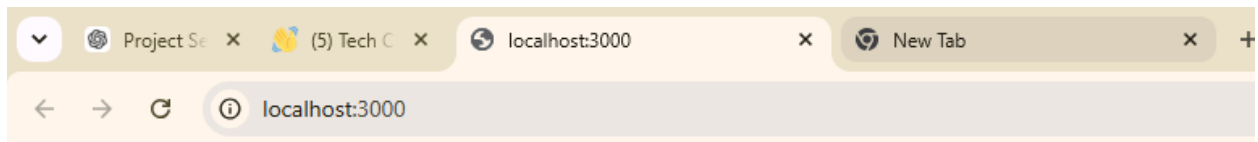
```
Running on http://127.0.0.1:3000
```

Verification / Browser Test

The application was successfully accessed in the browser at:

<http://localhost:3000>

Output:



Hello from CI/CD Pipeline!

Result

The application ran successfully on the local machine and was accessible through the browser on port 3000.

This confirmed:

- Python environment works
- Dependencies are installed
- Application works correctly before containerization

6. Dockerization

Objective

Containerize the Flask application so it can run consistently across different environments.

Why Docker

At this point, the application was already running successfully on the local machine. The next step was to package the application and its dependencies into a portable container image that could later be deployed to AWS.

Docker solves environment consistency issues by ensuring the application runs the same way locally, in CI/CD pipelines, and in production.

Dockerfile

```
FROM python:3.12-slim

WORKDIR /app

COPY app/requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY app/ .

EXPOSE3000

CMD ["python", "app.py"]
```

Build Docker Image

Command used:

```
docker build-t cicd-flask-app .
```

Result:

Docker successfully built the application image.

Run Docker Container

Command used:

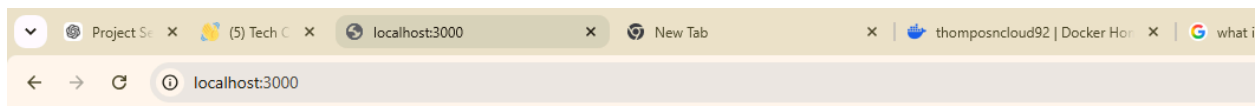
```
docker run -p3000:3000 cicc-flask-app
```

Verification

The container started successfully and the application was accessible in the browser at:

```
http://localhost:3000
```

Output:



Hello from CI/CD Pipeline!

Result

The application was successfully containerized and tested inside Docker.

This confirmed:

- Docker image builds successfully
- Container runs correctly
- Application is ready for cloud deployment

Challenge Encountered

Docker initially failed to build due to:

- Docker daemon not running
- Docker Hub authentication issue

These were resolved by:

- Starting Docker Desktop
- Refreshing Docker authentication

7. Infrastructure Provisioning with Terraform

Objective

Provision the AWS infrastructure required to deploy the containerized Flask application on ECS Fargate.

Why Terraform

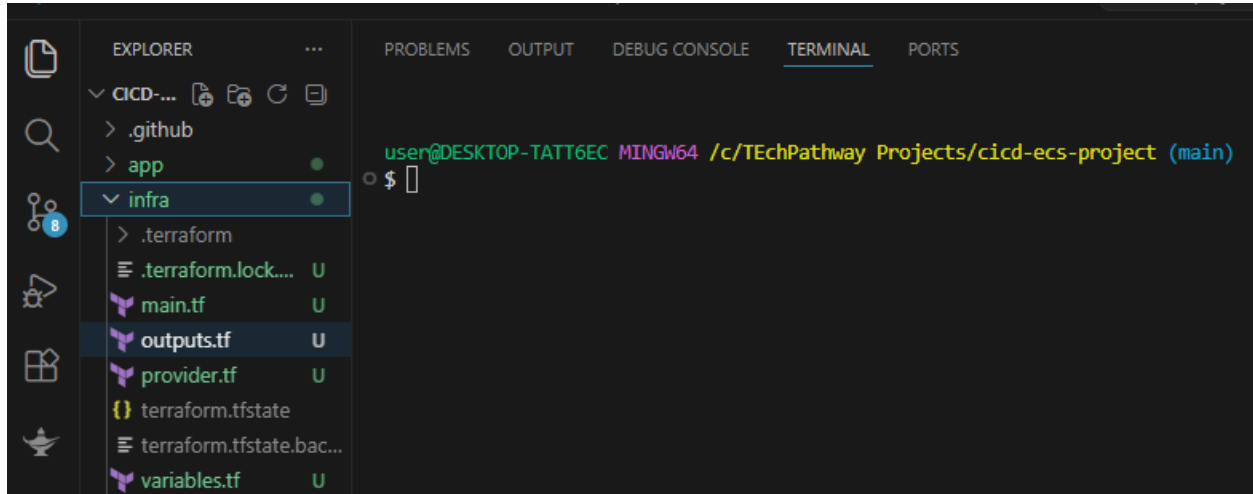
I used Terraform to provision the infrastructure instead of manually creating resources in AWS Console.

This made the setup easier to manage, repeatable, and version-controlled. It also made it easier to tear everything down after testing to avoid unnecessary AWS costs.

For this project, Terraform was responsible for provisioning both the networking layer and the application infrastructure.

Terraform Files

Infrastructure configuration was organized inside the `infra` directory.



Files created:

- `provider.tf` → AWS provider configuration
- `variables.tf` → Reusable variables
- `main.tf` → Infrastructure resources
- `outputs.tf` → Deployment outputs

Commands Used

```
terraform init
terraform plan
terraform apply
```

Infrastructure Provisioned

Terraform successfully created the following AWS resources:

- VPC
- Public Subnet 1

- Public Subnet 2
- Internet Gateway
- Route Table
- Security Groups
- Amazon ECR Repository
- ECS Cluster
- ECS Task Definition
- ECS Service
- Application Load Balancer
- Target Group
- Listener
- IAM Task Execution Role

Terraform plan output:

```
Plan:18 to add,0 to change,0 to destroy
```

Architecture Design

For this setup, I used ECS Fargate instead of EC2-backed ECS.

I chose Fargate because I wanted to focus on container orchestration and deployment automation without managing EC2 instances.

This setup works well for development and testing environments.

Traffic flow was designed as:

Internet → ALB → Target Group → ECS Service → Container

The ALB receives incoming HTTP traffic and forwards requests to healthy ECS tasks running inside the cluster.

Verification

After deployment, all resources were verified in AWS Console.

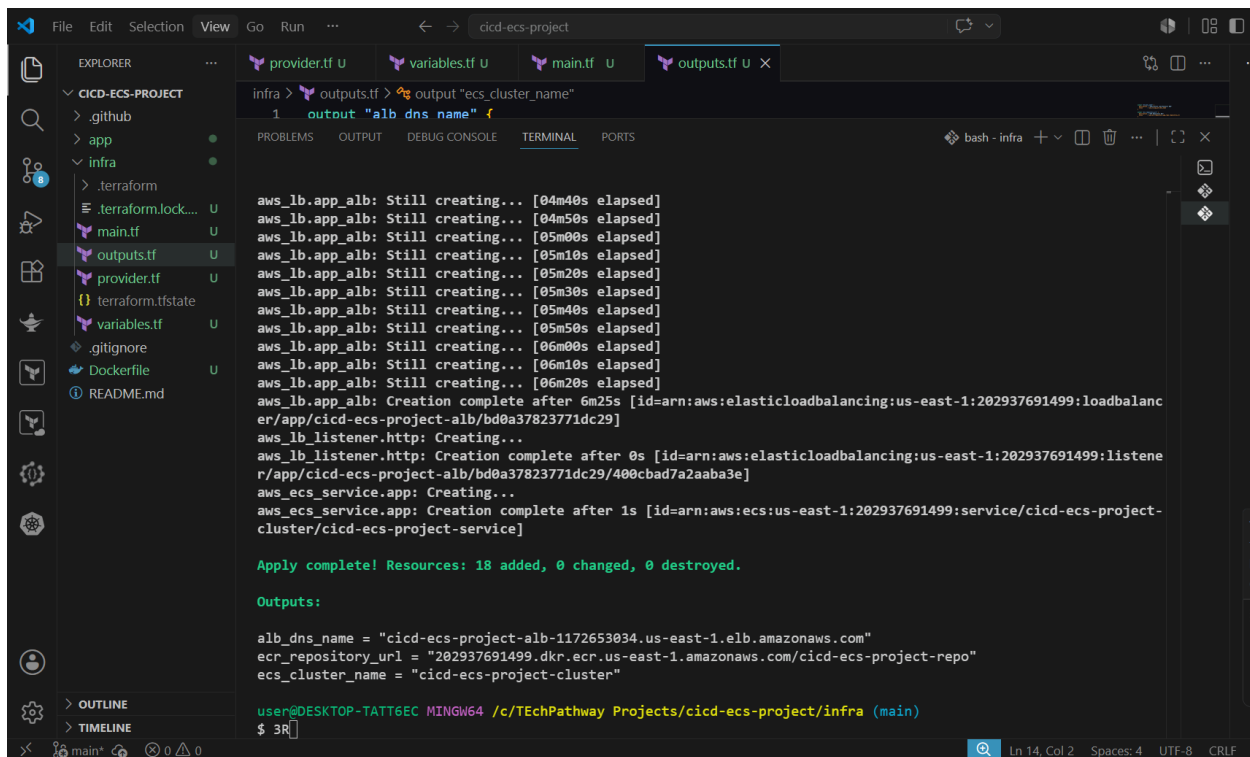
Confirmed resources included:

- ECS Cluster
- ECS Service
- ECR Repository
- Application Load Balancer
- Target Group

The ECS task registered successfully with the target group and passed health checks.

Screenshots:

- Terraform apply successful



```
infra > terraform apply -input=false
terraform apply -input=false
aws_lb.app_alb: Still creating... [04m40s elapsed]
aws_lb.app_alb: Still creating... [04m50s elapsed]
aws_lb.app_alb: Still creating... [05m00s elapsed]
aws_lb.app_alb: Still creating... [05m10s elapsed]
aws_lb.app_alb: Still creating... [05m20s elapsed]
aws_lb.app_alb: Still creating... [05m30s elapsed]
aws_lb.app_alb: Still creating... [05m40s elapsed]
aws_lb.app_alb: Still creating... [05m50s elapsed]
aws_lb.app_alb: Still creating... [06m00s elapsed]
aws_lb.app_alb: Still creating... [06m10s elapsed]
aws_lb.app_alb: Still creating... [06m20s elapsed]
aws_lb.app_alb: Creation complete after 6m25s [id=arn:aws:elasticloadbalancing:us-east-1:202937691499:loadbalancer/app/cicd-ecs-project-alb/bd0a37823771dc29]
aws_lb_listener.http: Creating...
aws_lb_listener.http: Creation complete after 0s [id=arn:aws:elasticloadbalancing:us-east-1:202937691499:listener/app/cicd-ecs-project-alb/bd0a37823771dc29/400cbad7a2aaba3e]
aws_ecs_service.app: Creating...
aws_ecs_service.app: Creation complete after 1s [id=arn:aws:ecs:us-east-1:202937691499:service/cicd-ecs-project-cluster/cicd-ecs-project-service]

Apply complete! Resources: 18 added, 0 changed, 0 destroyed.

Outputs:

alb_dns_name = "cicd-ecs-project-alb-1172653034.us-east-1.elb.amazonaws.com"
ecr_repository_url = "202937691499.dkr.ecr.us-east-1.amazonaws.com/cicd-ecs-project-repo"
ecs_cluster_name = "cicd-ecs-project-cluster"

user@DESKTOP-TATT6EC MINGW64 /c/TechPathway Projects/cicd-ecs-project/infra (main)
$
```

- ECS service running

- Healthy target group
 - Application accessible via ALB
-

To keep the setup simple, ECS tasks were deployed in public subnets with public IP assignment enabled.

This reduced networking complexity and avoided the need for NAT gateways.

Security groups were configured to ensure:

- ALB accepts HTTP traffic from the internet
- ECS tasks only accept traffic from the ALB

This ensured controlled communication between components.

8. Docker Image Push to Amazon ECR

Objective

Build the Docker image locally and push it to Amazon ECR so it can be pulled by ECS during deployment.

Commands Used

Build Docker image:

```
docker build -t cicd-ecs-project .
```

Authenticate Docker to ECR:

```
aws ecr get-login-password --region us-east-1 | docker login --  
username AWS --password-stdin <ECR_REPOSITORY_URL>
```

Tag Docker image:

```
docker tag cicd-ecs-project:latest <ECR_REPOSITORY_URL>:lates
```

t

Push image to ECR:

```
docker push <ECR_REPOSITORY_URL>:latest
```

Explanation

The Docker image was first built locally using the application source code and Dockerfile.

After authenticating with Amazon ECR, the image was tagged with the ECR repository URL and pushed to the registry.

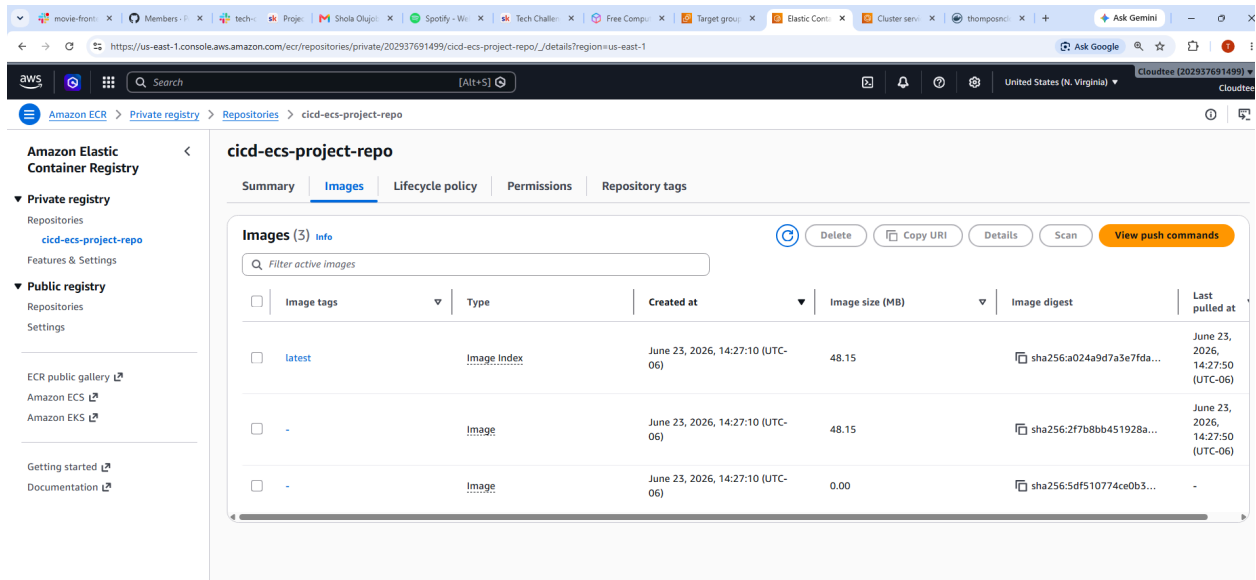
This made the application image available for ECS deployment.

Verification

The image push was verified in Amazon ECR.

Confirmed:

- Repository created successfully
- Image push completed successfully
- `latest` tag available in ECR



Tagging tells Docker exactly which registry and repository the image should be pushed to.

9. Application Deployment on AWS ECS

Objective

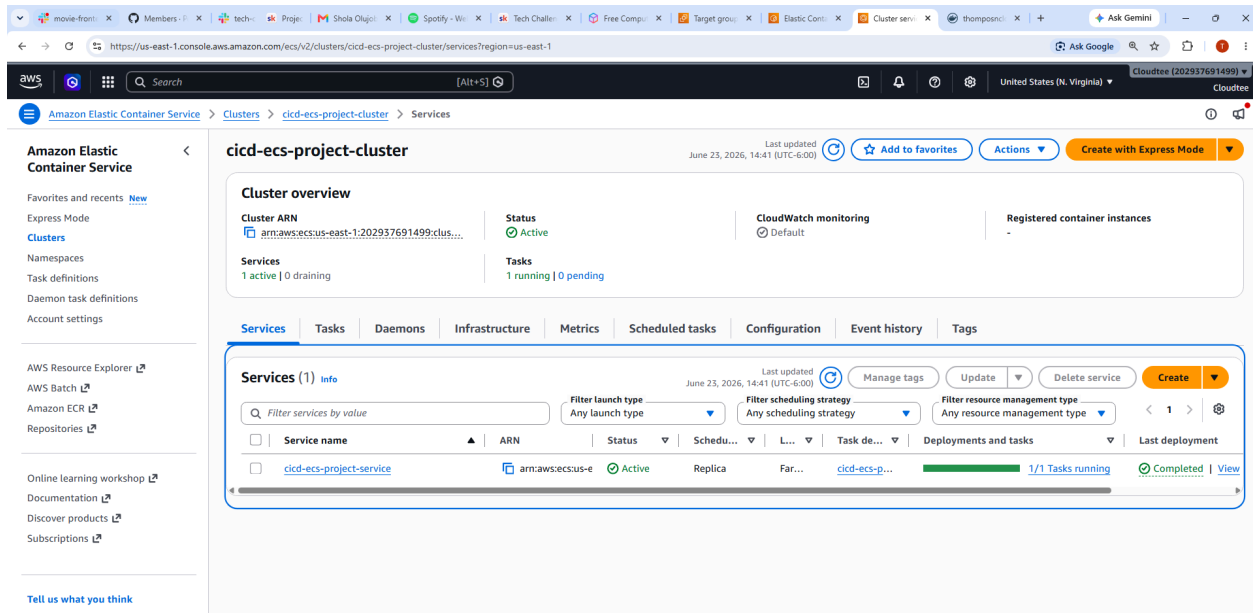
Deploy the containerized Flask application to AWS ECS using Fargate and expose it through an Application Load Balancer.

Deployment Process

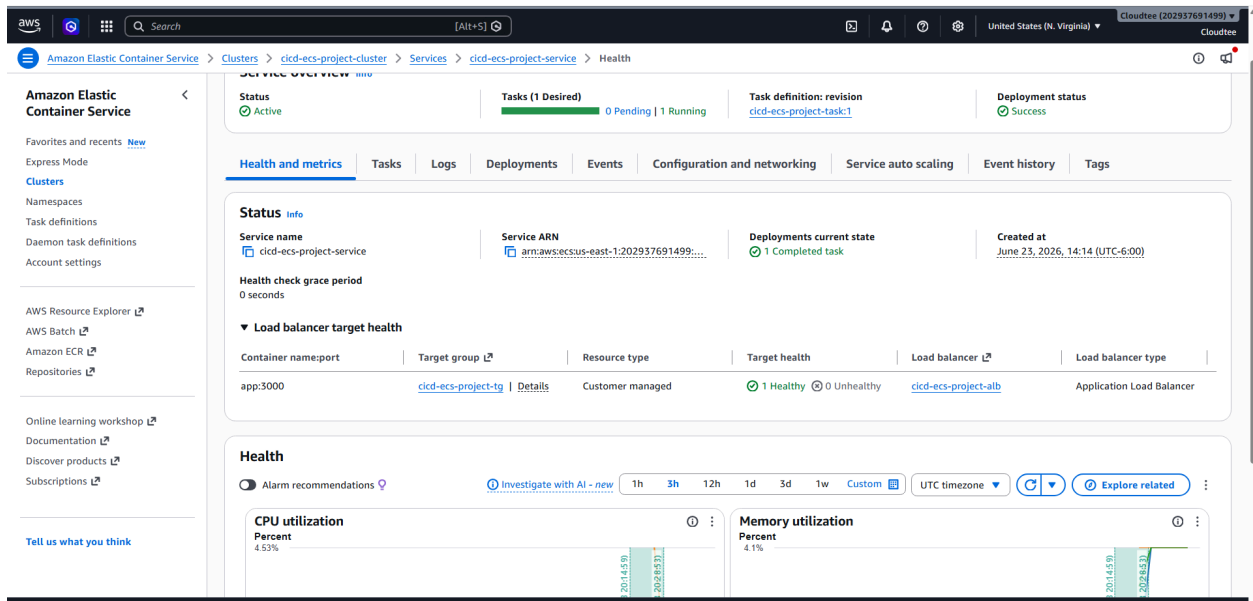
The application was deployed using the infrastructure provisioned by Terraform.

Core resources used during deployment:

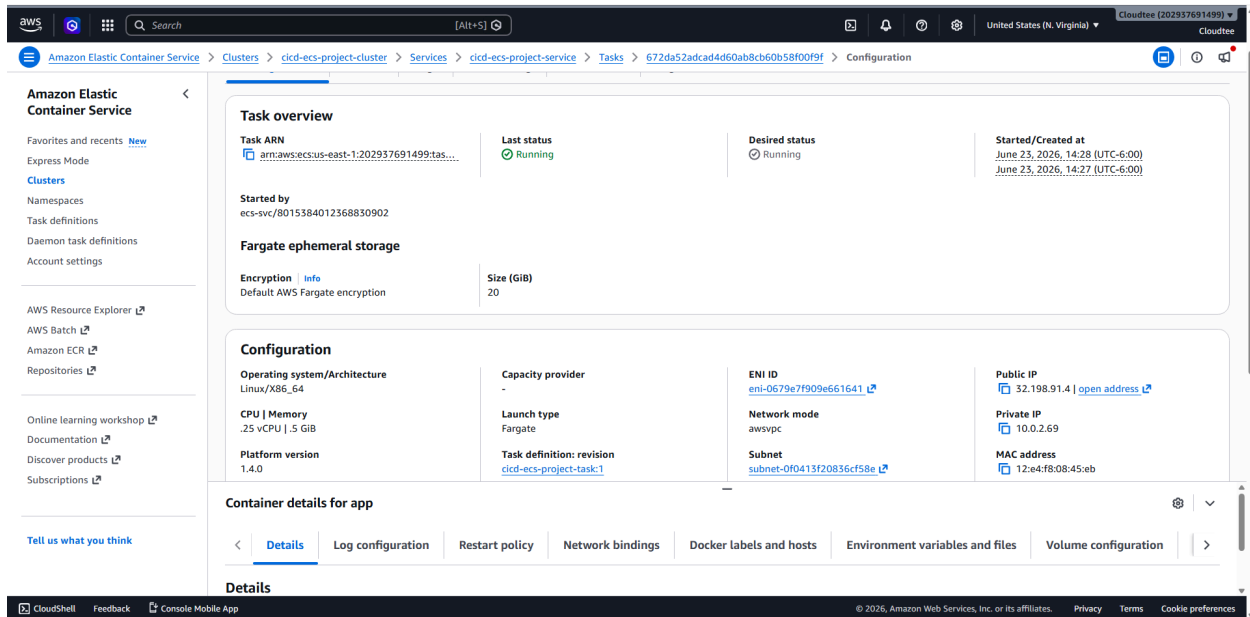
- ECS Cluster



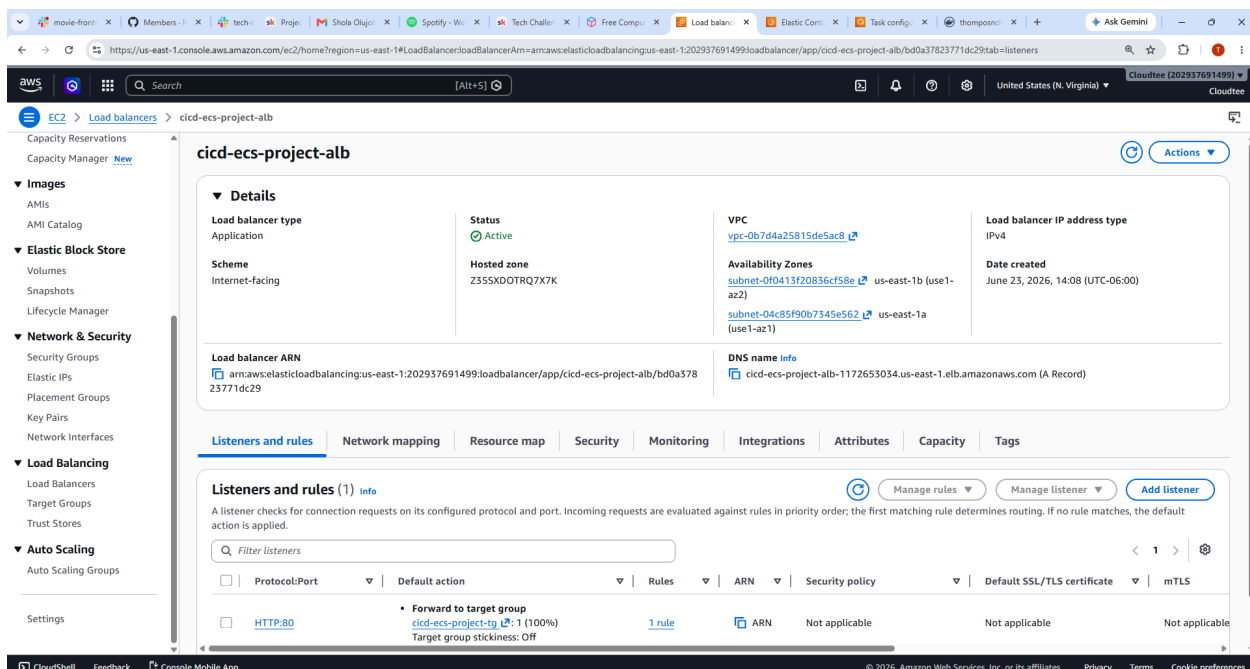
• ECS Service



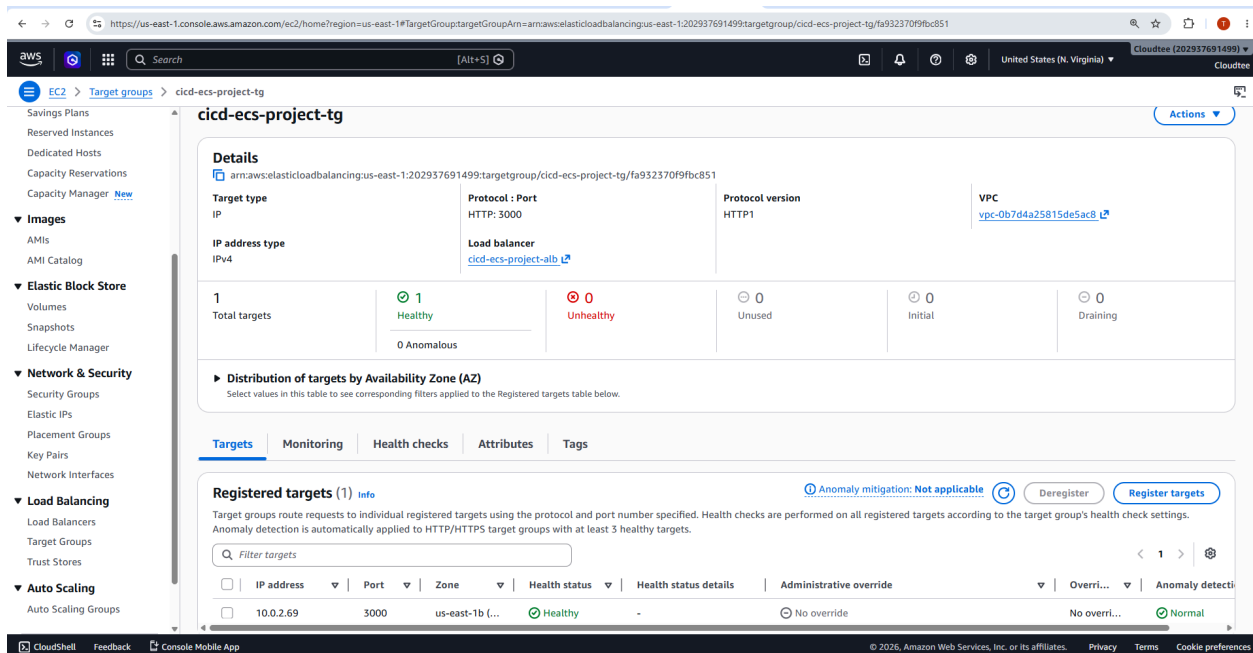
• ECS Task Definition



- Application Load Balancer



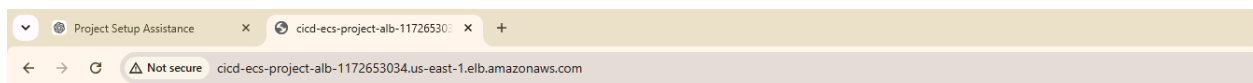
- Target Group



The ECS task definition referenced the Docker image stored in Amazon ECR. AWS Fargate was used as the launch type, removing the need to manage EC2 instances.

Verification

After deployment, all resources were verified in AWS Console.



Hello from CI/CD Pipeline!

Confirmed resources included:

- ECS Cluster
- ECS Service
- ECR Repository
- Application Load Balancer
- Target Group

The ECS task registered successfully with the target group and passed health checks.

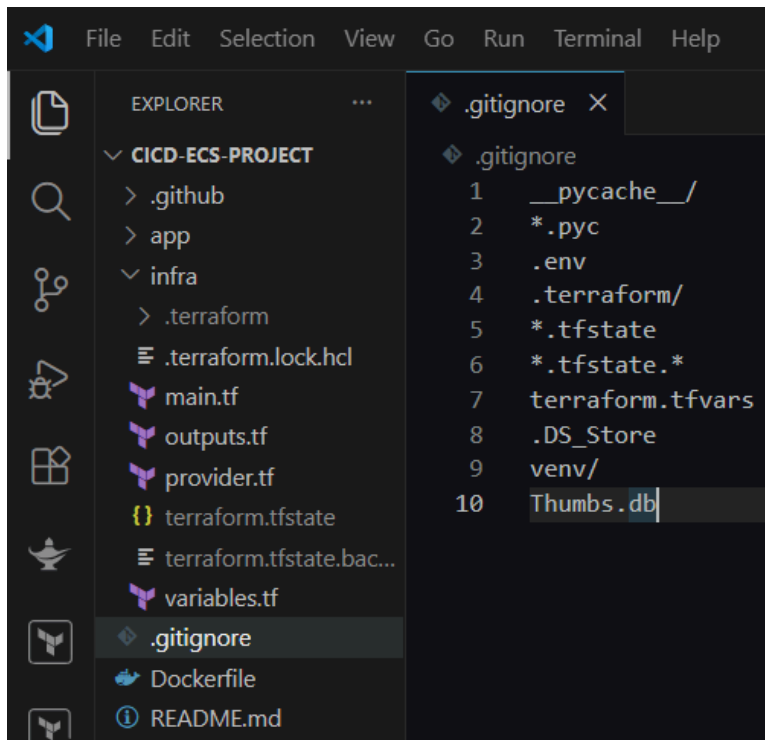
11. GitHub Repository Setup

Objective

Set up a GitHub repository to version-control the project and prepare it for CI/CD automation with GitHub Actions.

Git Ignore Configuration

A `.gitignore` file was configured to exclude unnecessary and sensitive files from version control.



This ensures Terraform state files, local environment files, and cache files are not committed.

Initial Commit and Push

The project files were staged and committed locally.

Commands used:

```
git add .
git commit -m "Initial CI/CD ECS project setup"
```

The local repository was then connected to GitHub and pushed to the `main` branch.

```
git remote add origin https://github.com/Thompson-Cloud/cicd-ecs-project.git
git push -u origin main
```

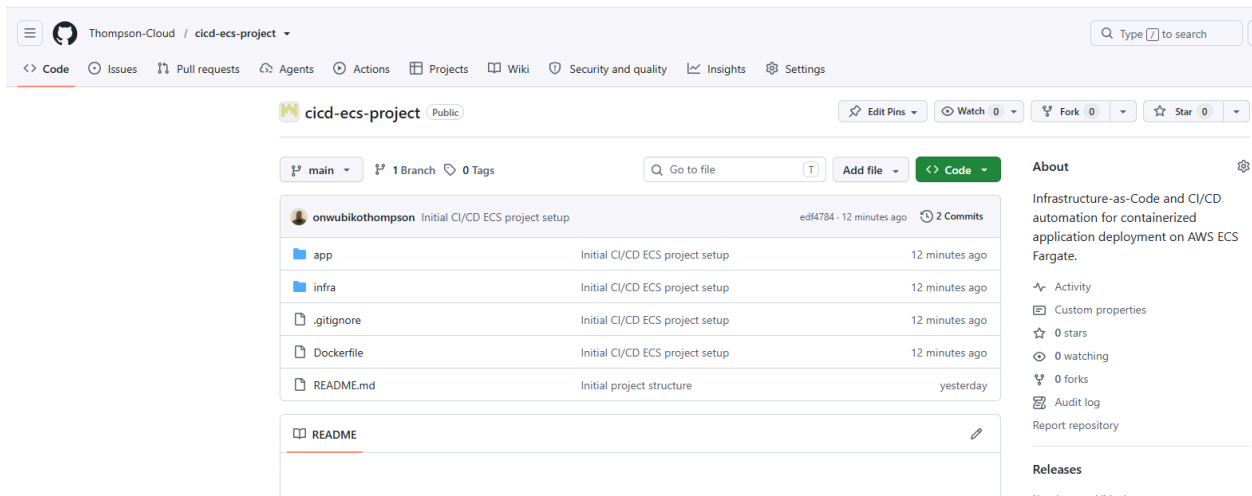
Repository Structure

Final project structure:

```
cicd-ecs-project/
├── .github/
├── app/
│   ├── app.py
│   └── requirements.txt
├── infra/
│   ├── provider.tf
│   ├── variables.tf
│   ├── main.tf
│   └── outputs.tf
├── .gitignore
├── Dockerfile
└── README.md
```

Verification

The repository was successfully pushed to GitHub with all required project files.



The screenshot shows the GitHub interface for a repository named 'cicd-ecs-project' by user 'onwubikothompson'. The repository is public and has 1 branch (main) and 0 tags. The commit history shows two commits: 'Initial CI/CD ECS project setup' (ed14784, 12 minutes ago) and 'Initial project structure' (yesterday). The file list includes 'app', 'infra', '.gitignore', 'Dockerfile', and 'README.md'. The 'About' section describes the repository as 'Infrastructure-as-Code and CI/CD automation for containerized application deployment on AWS ECS Fargate'.

Confirmed files included:

- Application code

- Dockerfile
- Terraform configuration
- GitHub workflow directory
- README

The repository is now ready for CI/CD pipeline implementation using GitHub Actions.

12. CI/CD Pipeline with GitHub Actions

Objective

Automate the application deployment workflow using GitHub Actions.

The pipeline is designed to:

- Trigger automatically on pushes to the `main` branch
- Build the Docker image
- Authenticate to AWS
- Push the image to Amazon ECR
- Deploy the updated container to Amazon ECS

This removes manual deployment steps and ensures consistent releases.

Workflow Architecture

Pipeline flow:

```
Developer Push → GitHub → GitHub Actions → Docker Build → ECR Push → ECS Deployment
```

Deployment process:

1. Developer pushes code to `main`
2. GitHub Actions workflow starts automatically

3. Docker image is built
 4. Image is pushed to Amazon ECR
 5. ECS service pulls the latest image
 6. ECS performs rolling deployment
-

Workflow Directory

The GitHub Actions workflow was created inside:

```
.github/workflows/
```

Workflow file:

```
deploy.yml
```

Trigger Configuration

The pipeline is configured to run automatically on pushes to the main branch.

```
on:
  push:
    branches:
      - main
```

This ensures every approved code change triggers deployment.

CI/CD Workflow Stages

The pipeline contains the following major stages:

- Checkout source code
- Configure AWS credentials
- Authenticate to Amazon ECR

- Build Docker image
 - Push image to ECR
 - Deploy updated image to ECS
-

AWS Authentication

GitHub Actions requires secure access to AWS resources.

Authentication is handled using GitHub Secrets and AWS credentials.

Required secrets:

- `AWS_ACCESS_KEY_ID`
- `AWS_SECRET_ACCESS_KEY`

These secrets are stored securely in GitHub repository settings.

11. GitHub Actions CI/CD Pipeline

Objective

Automate build and deployment so that every push to the `main` branch triggers a complete CI/CD workflow.

The workflow was implemented using GitHub Actions.

Pipeline stages include:

- Checkout source code
 - Configure AWS credentials
 - Authenticate to Amazon ECR
 - Build Docker image
 - Tag Docker image
 - Push Docker image to ECR
 - Deploy updated image to ECS
-

Workflow Trigger

The workflow runs automatically whenever code is pushed to the `main` branch.

```
on:
  push:
    branches:
      - main
```

This ensures deployments happen automatically without manual intervention.

GitHub Secrets

Sensitive AWS credentials were stored securely as GitHub repository secrets.

Configured secrets:

- `AWS_ACCESS_KEY_ID`
- `AWS_SECRET_ACCESS_KEY`

These secrets were used by GitHub Actions to authenticate with AWS.

GitHub Variables

Non-sensitive configuration values were stored as GitHub repository variables.

Configured variables:

- `AWS_REGION`
- `ECR_REPOSITORY`
- `ECS_CLUSTER`
- `ECS_SERVICE`

This improves maintainability by keeping environment configuration outside the workflow file.

Workflow File

The CI/CD workflow was created inside:

```
.github/workflows/deploy.yml
```

The workflow handles image build, image push, and ECS deployment.

Image Tagging Strategy

A SHA-based image tagging strategy was implemented to improve deployment traceability.

Example image tags:

```
27ef5eba5f62bc65882826710860b4cd1bcd4a75  
latest
```

Benefits of this approach:

- Every deployment maps to a specific commit
 - Easier rollback during failures
 - Better version tracking in production
-

Pipeline Verification

After configuration, GitHub Actions completed successfully.

Verified outcomes:

- Docker image built successfully
- Image pushed to Amazon ECR
- ECS service updated successfully

Pipeline status:

Thompson-Cloud / cicd-ecs-project

<> Code

Issues

Pull requests

Agents

Actions

Projects

Wiki

← Deploy to ECS

✓

Add GitHub Actions ECS deployment workflow #1

Summary

All jobs

✓

Build, Push, and Deploy

Run details

Usage

Workflow file

Annotations

1 warning

Build, Push, and Deploy

succeeded 46 minutes ago in 30s

> ✓ Set up job

> ✓ Checkout source code

> ✓ Configure AWS credentials

> ✓ Login to Amazon ECR

> ✓ Build Docker image

> ✓ Tag Docker image for ECR

> ✓ Push Docker image to ECR

> ✓ Deploy to ECS

> ✓ Post Login to Amazon ECR

> ✓ Post Configure AWS credentials

> ✓ Post Checkout source code

> ✓ Complete job

Continuous Deployment Validation

CI/CD Pipeline Documentation

27

To validate true CI/CD behavior, the application response message was updated and pushed to the `main` branch.

This triggered:

- Automatic GitHub Actions workflow execution
- Docker image rebuild
- New image push to Amazon ECR
- ECS service redeployment

This confirmed that application code changes were automatically deployed to production.

12. Deployment Verification

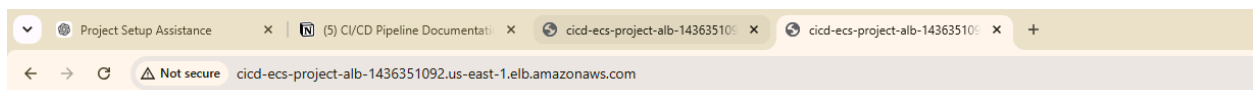
Application Load Balancer

After deployment, the application was accessed using the Application Load Balancer DNS.

URL:

```
http://cicd-ecs-project-alb-1436351092.us-east-1.elb.amazonaws.com
```

Final application response:



Congratulations Thompson! Your CI/CD pipeline is running successfully on AWS ECS.

This confirmed successful external access to the deployed container.

ECS Service Verification

The ECS service was verified in AWS Console.

Confirmed state:

- Service Status: Active
- Desired Tasks: 1
- Running Tasks: 1
- Deployment Status: Success

This confirmed the ECS service successfully deployed and maintained the application container.

Target Group Health Check

Load balancer target group health was verified.

Health status:

The screenshot shows the AWS Management Console interface for the 'cicd-ecs-project-tg' target group. The left sidebar contains navigation links for various AWS services. The main content area displays the target group configuration, including target type (IP), protocol (HTTP), and port (3000). It shows 1 total target, which is healthy. The 'Registered targets' table lists one target with IP 10.0.2.119, port 3000, in the us-east-1b zone, with a healthy status.

Target type	Protocol : Port	Protocol version	VPC
IP	HTTP: 3000	HTTP1	vpc-0b76670455d56af00

Target type	IP address type	Load balancer
IP	IPv4	cicd-ecs-project-alb

1	1	0	0	0	0
Total targets	Healthy	Unhealthy	Unused	Initial	Draining
0 Anomalous					

Distribution of targets by Availability Zone (AZ)
Select values in this table to see corresponding filters applied to the Registered targets table below.

Targets | Monitoring | Health checks | Attributes | Tags

Registered targets (1) [Info](#) [Anomaly mitigation: Not applicable](#) [Deregister](#) [Register targets](#)

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

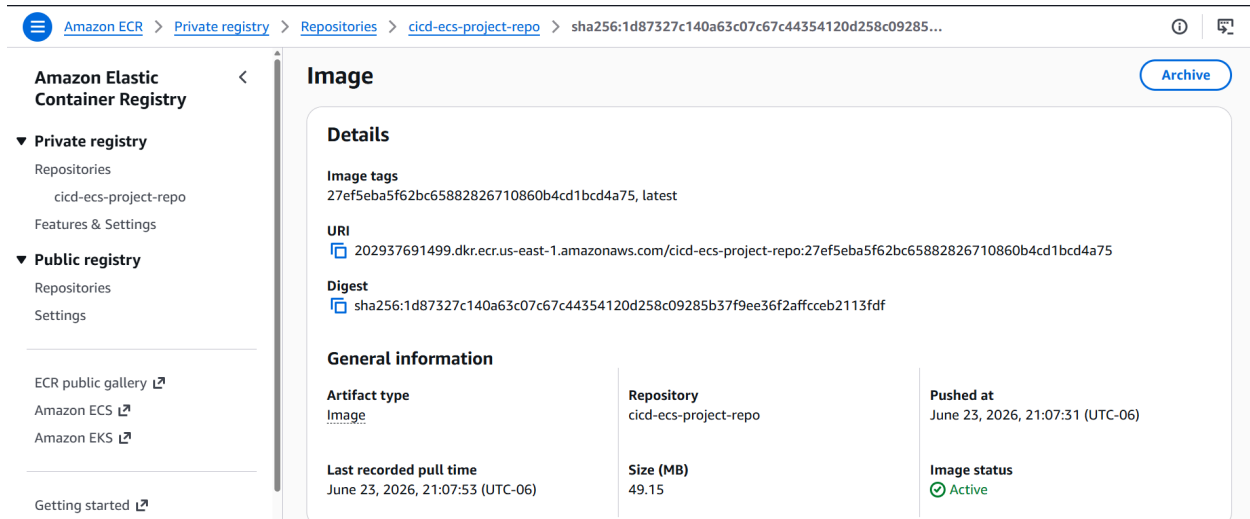
☐	IP address	Port	Zone	Health status	Health status details	Admini...	Overri...	Anomaly detectio...
☐	10.0.2.119	3000	us-east-1b (...)	Healthy	-	No override	No overri...	Normal

This confirmed the ECS task passed health checks and was receiving traffic successfully.

ECR Repository Verification

The Docker image was successfully pushed to Amazon ECR.

Verified image tags:



This confirmed successful image storage and deployment versioning.

13. Project Outcome and Lessons Learned

Project Outcome

This challenge successfully implemented a complete CI/CD pipeline for a containerized Flask application using AWS and GitHub Actions.

The final solution achieved:

- Application development using Flask
- Containerization using Docker
- Infrastructure provisioning using Terraform
- Container image management using Amazon ECR
- Automated deployment using GitHub Actions
- Production hosting using AWS ECS Fargate
- Automatic redeployment after application code changes

The full deployment lifecycle was successfully validated from code commit to production release.

Final verification confirmed successful automatic redeployment after application updates, proving that the CI/CD pipeline functions reliably in a production-like environment.

Challenges Encountered

Key challenges encountered during implementation included:

- Python environment setup on local machine
- Terraform state cleanup during infrastructure destroy
- ECR repository deletion due to retained images
- Pipeline failure when ECR repository was unavailable
- GitHub Actions deployment troubleshooting

Each issue was resolved through debugging, validation, and infrastructure re-verification.

Lessons Learned

Key takeaways from this project:

- Infrastructure must exist before CI/CD execution
- GitHub Secrets improve workflow security
- SHA-based image tagging improves deployment traceability
- Terraform simplifies infrastructure provisioning and recreation
- ECS Fargate enables efficient container orchestration without server management
- CI/CD pipelines should always be validated using real code changes

This project reinforced practical DevOps skills across application development, automation, infrastructure, and cloud deployment.

14. Cleanup

Infrastructure Cleanup

To avoid unnecessary AWS charges, infrastructure can be destroyed after project completion.

Command used:

```
terraform destroy
```

This removes all provisioned AWS resources including:

- VPC
- Subnets
- Security Groups
- Application Load Balancer
- ECS Cluster
- ECS Service
- Target Group

Important Note

Amazon ECR repositories containing images may not delete automatically during destroy.

If the repository contains images, either:

- Delete images manually from ECR
- or
- Configure Terraform with force delete enabled

This ensures complete infrastructure cleanup.